Building with Claude API

Complete System Design & API Patterns

Anthropic Official API Docs | docs.anthropic.com | April 2026

Overview

The Anthropic API provides access to Claude models with features including: streaming, vision, tool use, extended thinking, prompt caching, and batch inference. This document covers the API design with practical examples.

Source: docs.anthropic.com/en/api

1. API Authentication & Models

Authenticate with the x-api-key header. Current production models (April 2026):

Model	API String	Context	Best For
`Claude Opus 4.5`	`claude-opus-4-5`	200K	Complex reasoning, agents
`Claude Sonnet 4.6`	`claude-sonnet-4-6`	200K	Balanced perf/cost, everyday use
`Claude Haiku 4.5`	`claude-haiku-4-5-20251001`	200K	Fast, cheap, high volume

2. Basic API Call (Python SDK)

```
import anthropic

client = anthropic.Anthropic(api_key="your-key")  # or ANTHROPIC_API_KEY env var

message = client.messages.create(
    model="claude-sonnet-4-6",
    max_tokens=1024,
    system="You are a helpful AI assistant specializing in data analysis.",
    messages=[
        {"role": "user", "content": "Explain the difference between RAG and fine-tun
ing."}
    ]
)

print(message.content[0].text)
```

```
print(f"Input tokens: {message.usage.input_tokens}")
print(f"Output tokens: {message.usage.output_tokens}")
```

3. Streaming

```
# Stream responses for real-time display
with client.messages.stream(
    model="claude-sonnet-4-6",
    max_tokens=1024,
    messages=[{"role": "user", "content": "Write a poem about neural networks."}]
) as stream:
    for text in stream.text_stream:
        print(text, end="", flush=True)
```

4. Prompt Caching

Prompt caching reduces cost by up to 90% and latency by up to 85% when reusing large stable prefixes (system prompts, documents, few-shot examples). Mark cacheable content with cache_control:

```
# System prompt caching (charged at 0.1x write, free on cache hit)
response = client.messages.create(
    model="claude-sonnet-4-6",
    max_tokens=1024,
    system=[
        {
            "type": "text",
            "text": long_document_text,  # 50K token document
            "cache_control": {"type": "ephemeral"}  # Cache this for 5 min
        }
    ],
    messages=[{"role": "user", "content": "Summarize the key findings."}]
)
print(f"Cache hit: {response.usage.cache_read_input_tokens} tokens")
```

5. Extended Thinking

```
# Enable extended thinking for hard problems
response = client.messages.create(
    model="claude-sonnet-4-6",
    max_tokens=16000,
    thinking={"type": "enabled", "budget_tokens": 10000},
    messages=[{"role": "user", "content": "Solve: 100 prisoners problem. Explain opt
imal strategy."}]
)
# Content blocks: [ThinkingBlock(thinking=...), TextBlock(text=...)]
for block in response.content:
    if block.type == "thinking":
```

```
        print(f" {block.thinking[:200]}...")
    else:
        print(f"Answer: {block.text}")
```

6. Batch API (Async, 50% cost)

For large-scale offline processing, the Batch API processes up to 100K requests asynchronously at 50% cost with 24-hour completion SLA:

```
import anthropic

client = anthropic.Anthropic()

# Create a batch
batch = client.messages.batches.create(
    requests=[
        {"custom_id": f"doc-{i}", "params": {
            "model": "claude-haiku-4-5-20251001",
            "max_tokens": 256,
            "messages": [{"role": "user", "content": f"Classify: {doc}"}]
        }} for i, doc in enumerate(documents)
    ]
)

# Poll until complete
import time
while (batch := client.messages.batches.retrieve(batch.id)).processing_status == "in
_progress":
    time.sleep(60)

# Download results
for result in client.messages.batches.results(batch.id):
    print(result.custom_id, result.result.message.content[0].text)
```